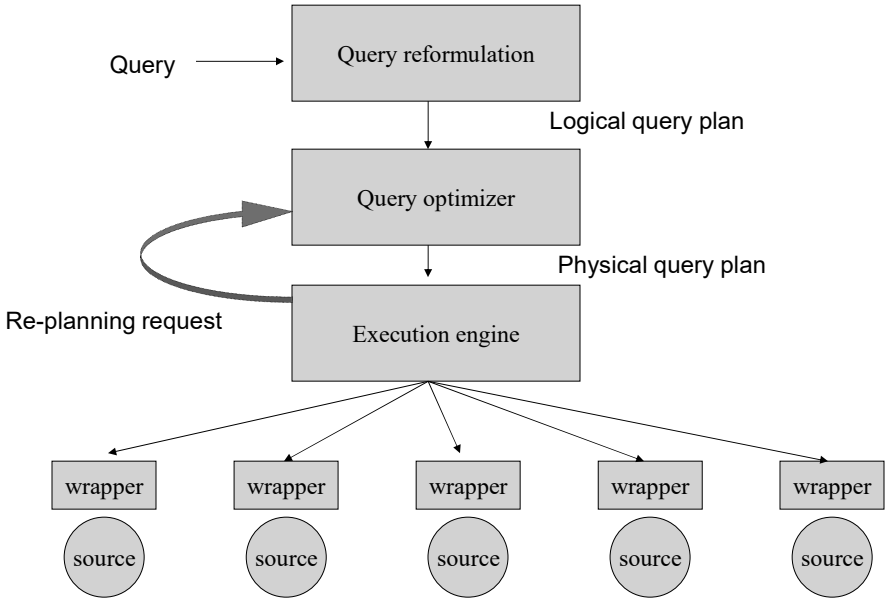
Sources Profiling & Selection

Vu Tuyet Trinh
SoICT - HUST

1



2

Wrapper

- A wrapper W extracts structured data from pages of S
- Formally, W is a tuple ($\mathbf{T}_W$, $\mathbf{E}_W$)
  - $\mathbf{T}_W$ is a target schema
    - this needs not be the same as the schema $\mathbf{T}_S$ used on the page, because we may want to extract only a subset of attributes of $\mathbf{T}_S$
  - $\mathbf{E}_W$ is an extraction program that uses format $\mathbf{F}_S$ to extract from each page a data instance conforming to $\mathbf{T}_W$
    - $\mathbf{E}_W$ is typically written in a script language (e.g., Perl) or in some higher-level declarative language that an execution engine can interpret

3

3

Example

- Consider a wrapper that extracts all attributes from pages of countries.com
  - target schema $\mathbf{T}_W$ is the source schema $\mathbf{T}_S$ = (country, capital, population, continent)
  - extraction program $\mathbf{E}_W$ may be a Perl script that specifies that given a page P from this source
    - return the first fully capitalized string as country
    - return the string immediately following “Capital:” as capital
    - etc.



countries.com

4

4

Source schema and page format

Schema T_S	**Format F_S**
Defines the logical fields and their nesting	Defines how the source displays those fields
Often remains implicit in a generated page	Uses tags, text labels and repeated regions
Example: (country, capital, population, continent)	Example: capital follows the label ‘Capital:’

5

Worked example: a book page

Page content	**Target and extraction**
Data Integration	T_W = Book(title, price)
<u>A. Author</u>	title: text within the book’s element
Price: $45.00	price: numeric text following ‘Price: $’
Publisher: Academic Press	Output: (‘Data Integration’, 45.00)

6

Two construction problems

Variant	Known input	Required output
Known target schema	Pages and desired fields	Extraction program E_W
Unknown source schema	Pages from one source	Schema T_S and program E_W

Learning the structure does not automatically reveal the meaning of its fields.

7

Construction challenges

- Inferring a grammar from positive page examples is difficult
- Even a known schema permits many candidate extraction programs
- Optional fields and repeated lists complicate boundaries
- Formatting exceptions can invalidate otherwise plausible rules
- A rule that fits training pages may fail on new pages

8

Main Solution Approaches

- Manual
  - developer manually creates $\mathbf{T}_W$ and $\mathbf{E}_W$
- Learning
  - developer highlights the attributes of $\mathbf{T}_W$ in a set of Web pages, then applies a learning algorithm to learn $\mathbf{E}_W$
- Automatic
  - automatically learn both $\mathbf{T}_S$ and $\mathbf{E}_W$ from a set of Web pages
- Interactive
  - combines aspects of learning and automatic approaches
  - developer provides feedback to a program until convergence

9

9

Wrapper construction approaches

Approach	Human input	Representative method
Manual	Extraction rules or code	Regex, DOM paths
Learning based	Target schema and labeled pages	HLRT, Stalker
Automatic	Unlabeled pages	RoadRunner
Interactive	Feedback during search	Co-testing, Poly, Lixto

Beautiful Soup, Pandas, Selenium, Scrapy
Instant Data Scraper, Firecrawl,

10

Manual extraction: strings and DOM

String model	DOM model
Treat the page as a sequence of characters or tokens	Treat HTML as a tree of elements
Use delimiters or regular expressions	Use paths to locate a field or record
Works when local boundaries remain stable	Uses structural context around values
Can fail when the same tag appears in unrelated regions	Can fail when a layout change alters the path

11

Worked example: DOM paths

Field	Path from the chapter's example	Extracted value
title	/html/body/div[1]/table/td[2]/text()	Inception
rating	/html/body/div[2]/table/td[2]/text()	8.4
runtime	/html/body/div[2]/table/td[4]/text()	148 mins

An inserted div or table cell can shift a positional path.

12

HLRT: a delimiter model

Head-left-right-tail represents extraction with string delimiters

For n attributes, specify 2n + 2 strings

W = (h, t, l_1, r_1, …, l_n, r_n)

h and t bound the page's data region

l_i and r_i bound values of attribute i

13

HLRT worked example

HTML fragment	**Wrapper parameters**
Calling codes<P>	h = <P> t = <HR>
<B>Australia</B> <I>61</I> 	l_1 = <B> r_1 = </B>
<B>East Timor</B> <I>670</I> 	l_2 = <I> r_2 = </I>
<HR>Copyright	Output: (Australia, 61), (East Timor, 670)

14

Learning HLRT parameters

- Label the start and end of every required value on sample pages
- Find candidate h and t strings in shared page boundaries
- Find candidate left delimiters as common suffixes before values
- Find candidate right delimiters as common prefixes after values
- Search for a parameter combination consistent with all labels

15

HLRT limitations

The model assumes a flat tuple schema

All fields require reliable delimiting strings

Nested lists do not fit the simple sequential model

Mixed formats make common delimiters hard to identify

Stalker introduces hierarchical extraction and richer landmarks

16

Stalker: nested tuple schemas

Node	Type	Example content
restaurant	Tuple	name, cuisine, addresses
addresses	List of tuples	One or more locations
address	Tuple	street, city, state, zip, phone
city	Scalar	Phoenix

Extraction descends through the schema, using each parent's text as context.

17

Stalker rules and landmarks

Commands

SkipTo(x): scan forward through landmark x

BackTo(x): scan backward through x

BackUntil(x): stop at x without consuming it

A sequence of commands narrows the context

Landmarks

Literal tokens: <b>, Cuisine:, comma

Token classes: Numeric, HTMLTag, Punctuation

Rules can include ordered alternatives

Start and end rules define a substring

18

Stalker extraction trace

Step	Context	Result
1	Complete restaurant page	name and cuisine
2	Complete restaurant page	Address-list substring
3	Address-list substring	Individual address substrings
4	One address substring	street, city, state, zip, phone

A child rule sees its parent's extracted region rather than the entire page.

19

Learning Stalker with sequential covering

Use pages with labeled instances of the schema nodes

Learn a rule that covers some training examples correctly

Remove those covered examples and learn another rule

Combine learned rules as a disjunction

Refine candidates by extending landmarks or adding commands

20

Automatic learning with RoadRunner

Input: unlabeled pages generated by the same source

Output: a schema and a matching extraction program

Initialize the program with one page

Generalize it to match each additional page

Learn structural fields with generic names such as A and B

21

RoadRunner mismatch types

Mismatch	Interpretation	Generalization
Different text	Variable field value	#PCDATA
Extra structural region	Optional content	(pattern)?
Different repeated counts	List or iterator	(pattern)+

For a tag mismatch, the algorithm tests an iterator before an optional.

22

RoadRunner worked generalization

Observed pages	Generalized pattern
Page 1: topic ‘Database’, no image, two books	Topic: <B>#PCDATA</B>
Page 2: topic ‘Data Integration’, image, three books	Optional image: (<IMG …/>)?
Topics and book titles differ	Book list: (<LI>…#PCDATA…</LI>)+
The number of book entries differs	Nested repetitions may require recursive matching

23

Interactive construction

Maintain candidate wrappers and detect uncertainty

Ask the user for feedback on a useful page or extraction result

Use the response to eliminate or refine candidates

Repeat until the wrapper satisfies the user

The key design choice is when to ask and what to ask

24

Interactive systems compared

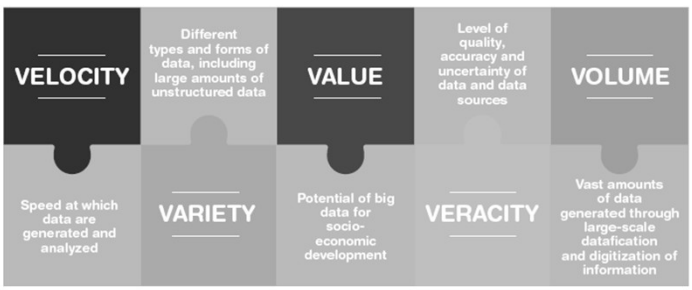
System	User interaction	Search mechanism
Stalker co-testing	Label a page with disagreement	Compare forward and backward rules
Poly	Choose the correct extracted tuples	Narrow DOM-based candidates
Lixto	Highlight records and fields	Build datalog-like extraction rules

25

Big Data Integration

- Questions ?
  - Source profiling: What does each source contain, and how suitable is its data?
  - Source selection: Which sources should we integrate, given our objectives and budget?

➢Dynamic source selection
➢Source summarization
➢Profiling at scale



26

Source profiling vs. Source selection

Aspect	Source profiling	Source selection
Main purpose	Understand & characterize sources	Choose a useful subset of sources
Input	Data, schemas, metadata, update history	Source profiles, integration objectives, costs
Main activities	Discover types, missing values, keys, dependencies, semantics and quality	Compare coverage, accuracy, freshness, overlap and marginal value
Output	A profile describing each source	Selected sources and, for dynamic data, acquisition frequencies
Main challenge	Scale, heterogeneous formats and unclear semantics	Source dependencies, uncertain benefits and changing quality

27

Big Data Integration emerging topics

- Crowdsourcing
- Source selection
- Source profiling

28

Crowdsourcing in integration

- Humans contribute judgments that remain difficult to automate
- Applications include schema alignment and record linkage
- The system must recruit and retain contributors
- It must combine contributions into a solution
- It must evaluate contributors and the quality of their work

29

Hybrid record linkage

- Naive pairwise labeling requires $O(n^2)$ human tasks
- Computational filtering removes unlikely matches
- The crowd labels the remaining candidate pairs
- Transitive relations can infer additional pair labels
- The labeling order changes how many human tasks remain

30

Valid and invalid transitive inference

Known labels	Inference	Status
A = B and B = C	A = C	Valid
A = B and B ≠ C	A ≠ C	Valid
A ≠ B and B ≠ C	A ? C	No inference

These rules assume consistent identity labels. Incorrect labels can propagate.

31

Transitivity example: candidate pairs

Pair	Likelihood	True label
L1: (R2, R3)	0.85	Match
L2: (R1, R2)	0.75	Match
L3: (R1, R6)	0.72	Non-match
L4: (R1, R3)	0.65	Match
L5: (R4, R5)	0.55	Match
L6: (R4, R6)	0.48	Non-match
L7: (R2, R4)	0.45	Non-match
L8: (R5, R6)	0.42	Non-match

Likelihood order: L1, L2, L3, L4, L5, L6, L7, L8

After L1 and L2, infer L4 as a match

After L5 and L6, infer L8 as a non-match

Six human labels suffice in this example

Parallel round 1: L1, L2, L3, L5, L6. Round 2: L7

32

Crowd labeling: evidence and limits

Chapter-reported result	Setting	Interpretation
95% fewer labeled pairs	Cora simulation	Many matching records per entity
About 20% fewer pairs	Abt-Buy simulation	Fewer multi-record entities
1,237 to 14 iterations	Cora, threshold 0.3	Parallel scheduling reduced rounds
96.5% fewer HITs, ~5% quality loss	Cora on AMT	Wrong labels propagated through inference

33

Corleone: end-to-end crowdsourcing

Module	Purpose
Blocker	Learn rules that remove likely non-matching pairs
Matcher	Train a random forest with crowd labels
Accuracy estimator	Estimate precision and recall despite class imbalance
Difficult pairs locator	Identify pairs needing further matching work

User input: two tables, matching instructions, two positive and two negative examples.

34

Corleone learning and evaluation

- Similarity features represent candidate pairs
- Active learning targets useful crowd labels
- Negative forest rules provide candidate blocking rules
- Accuracy estimation must handle rare positive pairs
- High-precision rules isolate easy cases so difficult pairs remain

35

Source selection: benefit and cost

- A new source can increase coverage or improve fusion accuracy
- A redundant source may add little benefit
- A low-quality source can reduce integrated-data accuracy
- Acquisition, engineering and computation all incur cost
- Source selection compares subsets before full integration

36

Source selection objectives

Objective	Constraint
Maximize benefit_I(S)	cost_I(S) ≤ budget
Minimize cost_I(S)	benefit_I(S) ≥ target
Maximize benefit_I(S) − cost_I(S)	cost_I(S) ≤ budget

The profit objective requires benefit and cost in comparable units.

37

Marginal benefit and greedy limitations

- Marginal benefit depends on the sources already selected
- Sources may overlap or complement one another
- Diminishing returns need not hold for data fusion
- Stopping at one marginal point may miss a better subset
- The general static selection problem is NP-complete

38

GRASP for static source selection

- Construct a candidate subset with randomized choices
- Choose from promising candidates rather than one fixed order
- Improve the subset using local search
- Repeat construction and search several times
- Return the best subset found across repetitions

39

Worked example: source value

Selected subset	Benefit ($)	Cost ($)	Profit ($)
A	100	40	60
B	80	30	50
C	60	20	40
A + B	130	70	60
A + C	150	60	90
B + C	120	50	70
A + B + C	160	90	70

With a $70 budget, A + C gives the largest profit among these subsets.

40

Dynamic sources: freshness

Update frequency

How often the source changes its contents

Frequent additions can coexist with stale old records

A daily update does not guarantee current values

Freshness

How well source values reflect the changing world

Requires detecting changes and deleting obsolete records

The best source subset can change over time

41

Time-aware selection

- Model coverage, accuracy and freshness over time
- Estimate changes in the world and in individual sources
- Choose a source subset and acquisition frequencies
- Some instances admit submodular optimization methods
- Approximation results require the stated objective and cost conditions

42

Source selection: modeling risks

- Copying and correlations weaken independence assumptions
- Estimated quality may differ from realized fusion quality
- Changes in acquisition frequency can alter cost and freshness
- Schema alignment and record linkage also affect integration value
- Re-evaluate the selected subset as sources evolve

43

Source profiling

- Profiling helps users understand an unfamiliar source
- Associate attributes with concepts, entities and relationships
- Quantify quality for relevant portions of the source
- Discover useful columns and potential join relationships
- Profiling supplies evidence for later source selection

44

Bellman: concise attribute summaries

- Large databases may have thousands of poorly documented tables
- Stored foreign keys and documentation may be incomplete
- Bellman summarizes column values with compact signatures
- Min-hash estimates overlap between sets of values
- q-gram and multiset summaries capture textual similarity and counts

45

Min-hash resemblance

$\rho(A, B) = |A \cap B| / |A \cup B|$

For hash function h_i: $s_i(A) = \min h_i(a)$, over a in A

Estimate ρ by the fraction of equal signature components

The estimate avoids repeatedly scanning all column values

Higher overlap suggests a candidate relationship to investigate

46

Worked example: attribute resemblance

Quantity	Value
A	{1, 2, 3, 4}
B	{2, 3, 4, 5}
Intersection size	3
Union size	5
Exact resemblance	3 / 5 = 0.60
Matching components in a toy 10-component signature	6 / 10 = 0.60

47

Schema summarization

- Represent tables and join relationships as a schema graph
- Cluster tables into k groups
- Choose a representative table for each group
- Use representatives as labels for a compact schema summary
- Table size or degree alone can misrepresent importance

48

Importance, strength and clustering

Component	Model
Table information	Sum of attribute entropies
Table importance	Stable information distribution on the schema graph
Join-path strength	Product of join-edge strengths
Strength between tables	Maximum strength among connecting paths
Table distance	1 − strength
Clustering	Greedy weighted k-center

49



50